

Lab 1

WITH ranked_films AS (

```
    SELECT c.name AS category, f.title, f.length,  
           DENSE_RANK() OVER (PARTITION BY c.name ORDER BY f.length DESC) AS rnk  
    FROM film f
```

```
    JOIN film_category fc ON f.film_id = fc.film_id
```

```
    JOIN category c ON fc.category_id = c.category_id
```

)

```
SELECT * FROM ranked_films WHERE rnk <= 3;
```

The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' showing the database structure. The main window displays a SQL query in the 'Query' tab. Below the query is the 'Data Output' tab, which shows the results of the query in a table format. The table has four columns: 'category', 'title', 'length', and 'rnk'. The results are sorted by 'rnk' in ascending order, showing the top 3 ranked films for each category.

category	title	length	rnk
Action	Worst Banger	185	1
Action	Dam Forrester	185	1
Action	Casualties Encino	179	2
Action	Quest Mussolini	177	3
Animation	Pond Seattle	185	1
Animation	Gangs Pride	185	1
Animation	Theory Mermaid	184	2
Animation	Sons Interview	184	2
Animation	Lawless Vision	181	3
Children	Wrong Behavior	178	1
Children	Fury Murder	178	1
Children	Empire Malkovich	177	2
Children	Model Fish	175	3
Classice	Conspirator Spirit	184	1

Lab 2

WITH monthly AS (

```
    SELECT DATE_TRUNC('month', payment_date) AS month, SUM(amount) AS revenue  
    FROM payment  
    GROUP BY 1
```

)

```
SELECT month, revenue,
```

```
       LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
```

```
       ROUND((revenue - LAG(revenue) OVER (ORDER BY month))
```

```
             / LAG(revenue) OVER (ORDER BY month) * 100, 2) AS growth_pct
```

```
FROM monthly
```

```
ORDER BY month;
```

The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' pane showing the database structure for 'PostgreSQL 16', including 'dvdrental' and 'public' schemas. The main area displays a SQL query in the 'Query' pane, which is joined with 'category' and calculates monthly revenue and growth percentage. Below the query is the 'Data Output' pane showing the results of the query.

Query:

```

6 JOIN category c ON fc.category_id = c.category_id
7 )
8 SELECT * FROM ranked_files WHERE rnk <= 3;
9
10 WITH monthly AS (
11     SELECT DATE_TRUNC('month', payment_date) AS month, SUM(amount) AS revenue
12     FROM payment
13     GROUP BY 1
14 )
15 SELECT month, revenue,
16        LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
17        ROUND((revenue - LAG(revenue) OVER (ORDER BY month))
18              / LAG(revenue) OVER (ORDER BY month) * 100, 2) AS growth_pct
19 FROM monthly
20 ORDER BY month;

```

Data Output:

	month timestamp without time zone	revenue numeric	prev_revenue numeric	growth_pct numeric
1	2007-02-01 00:00:00	8351.84	[null]	[null]
2	2007-03-01 00:00:00	23886.56	8351.84	186.00
3	2007-04-01 00:00:00	28559.46	23886.56	19.56
4	2007-05-01 00:00:00	514.18	28559.46	-98.20

Total rows: 4 Query complete 00:00:00.075

Lab 3

```

SELECT customer_id, payment_date, amount,
       SUM(amount) OVER (
         PARTITION BY customer_id
         ORDER BY payment_date
       ) AS running_clv
FROM payment
ORDER BY customer_id, payment_date;

```

The screenshot shows a PostgreSQL IDE interface. On the left is the Object Explorer showing the database structure. The main window displays two SQL queries. The first query calculates monthly revenue and growth percentage. The second query calculates the running total of payment amounts for each customer. Below the queries, the 'Data Output' tab shows the results of the second query as a table with 14 rows and 6 columns.

Query 1:

```

14 )
15 SELECT month, revenue,
16        LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
17        ROUND((revenue - LAG(revenue) OVER (ORDER BY month))
18              / LAG(revenue) OVER (ORDER BY month) * 100, 2) AS growth_pct
19 FROM monthly
20 ORDER BY month;

```

Query 2:

```

22 SELECT customer_id, payment_date, amount,
23        SUM(amount) OVER (
24          PARTITION BY customer_id
25          ORDER BY payment_date
26        ) AS running_clv
27 FROM payment
28 ORDER BY customer_id, payment_date;

```

Data Output Table:

	customer_id smallint	payment_date timestamp without time zone	amount numeric (5,2)	running_clv numeric
1	1	2007-02-14 23:22:38.996577	5.99	5.99
2	1	2007-02-15 16:31:19.996577	0.99	6.98
3	1	2007-02-15 19:37:12.996577	9.99	16.97
4	1	2007-02-16 13:47:23.996577	4.99	21.96
5	1	2007-02-18 07:10:14.996577	4.99	26.95
6	1	2007-02-18 12:02:25.996577	0.99	27.94
7	1	2007-02-21 04:53:11.996577	3.99	31.93
8	1	2007-03-01 07:19:30.996577	4.99	36.92
9	1	2007-03-02 14:05:18.996577	3.99	40.91
10	1	2007-03-02 16:30:04.996577	0.99	41.90
11	1	2007-03-17 11:06:20.996577	4.99	46.89
12	1	2007-03-18 02:25:55.996577	0.99	47.88
13	1	2007-03-19 08:23:42.996577	0.99	48.87
14	1	2007-03-19 12:25:20.996577	2.00	51.86

Summary: Total rows: 14596, Query complete 00:00:00.123. Status: Successfully run. Total query runtime: 123 msec. 14596 rows affected.

Lab4

```

WITH customer_spend AS (
  SELECT cu.customer_id, cu.first_name, cu.last_name, co.country,
         SUM(p.amount) AS total_spend
  FROM customer cu
  JOIN address a ON cu.address_id = a.address_id
  JOIN city ci ON a.city_id = ci.city_id
  JOIN country co ON ci.country_id = co.country_id
  JOIN payment p ON cu.customer_id = p.customer_id
  GROUP BY cu.customer_id, cu.first_name, cu.last_name, co.country
),
ranked AS (
  SELECT *, RANK() OVER (PARTITION BY country ORDER BY total_spend DESC) AS rnk
  FROM customer_spend
)
SELECT * FROM ranked WHERE rnk = 1
ORDER BY country;

```

Object Explorer

Servers

PostgreSQL 16

Databases (2)

dvdrental

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas (1)

public

Aggregates

Collations

Domains

FTS Configurations

FTS Dictionaries

FTS Parsers

FTS Templates

Foreign Tables

Functions

Materialized Views

Operators

Procedures

Sequences

Tables (15)

actor

address

category

city

country

customer

Columns

Constraints

Indexes

Dashboard X

dvdrental/postgre...

dvdrental/postgres@PostgreSQL 16* X

dvdrental/postgres@PostgreSQL 16

Query

Query History

Scratch Pad X

```
29
30 WITH customer_spend AS (
31   SELECT cu.customer_id, cu.first_name, cu.last_name, co.country,
32          SUM(p.amount) AS total_spend
33   FROM customer cu
34   JOIN address a ON cu.address_id = a.address_id
35   JOIN city ci ON a.city_id = ci.city_id
36   JOIN country co ON ci.country_id = co.country_id
37   JOIN payment p ON cu.customer_id = p.customer_id
38   GROUP BY cu.customer_id, cu.first_name, cu.last_name, co.country
39 ),
40 ranked AS (
41   SELECT *, RANK() OVER (PARTITION BY country ORDER BY total_spend DESC) AS rnk
42   FROM customer_spend
43 )
44 SELECT * FROM ranked WHERE rnk = 1
45 ORDER BY country;
```

Data Output

Messages

Notifications

Showing rows: 1 to 108

Page No: 1 of 1

	customer_id	first_name	last_name	country	total_spend	rnk
	integer	character varying (45)	character varying (45)	character varying (50)	numeric	bigint
1	218	Vera	Mccoy	Afghanistan	67.82	1
2	176	June	Carroll	Algeria	151.68	1
3	320	Anthony	Schwab	American Samoa	47.85	1
4	528	Claude	Herzog	Angola	93.80	1
5	381	Bobby	Boudreau	Anguilla	99.68	1
6	560	Jordan	Archuleta	Argentina	129.71	1
7	41	Stephanie	Mitchell	Armenia	118.75	1
8	266	Nora	Herrera	Austria	113.74	1
9	334	Raymond	McWhorter	Azerbaijan	108.75	1
10	590	Seth	Hannon	Bahrain	108.76	1
11	21	Michelle	Clark	Bangladesh	146.68	1
12	144	Clara	Shaw	Belarus	189.60	1

Total rows: 108 Query complete 00:00:00.142

LF Ln 43, Col 2